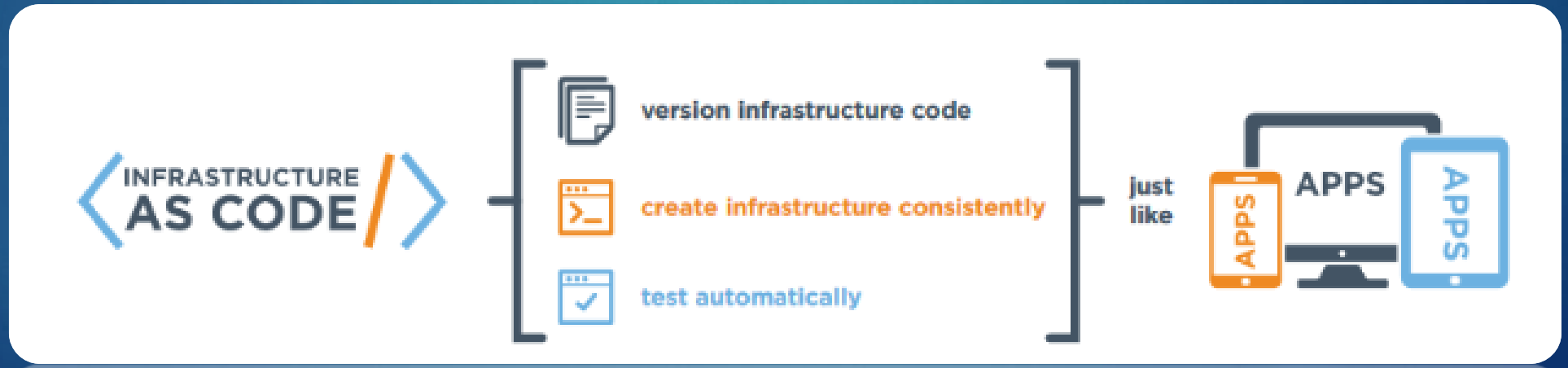
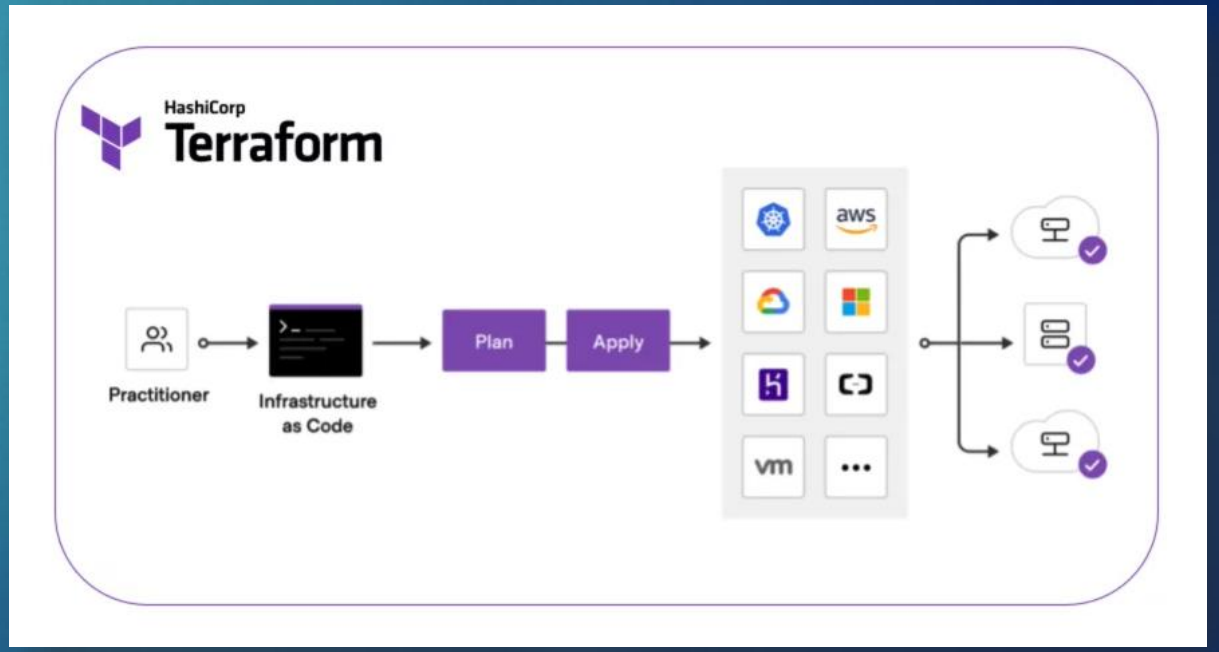
RECAP

- Infrastructure as Code (IaC) is an approach to infrastructure automation based on practices from software development
- IaC helps you automate the infrastructure deployment process in a repeatable, consistent manner, which has many benefits
- Remember…it's just code!
  - You can author it with any code editor and check it into a version control system along with your application code



RECAP

- Terraform is an Infrastructure as Code (IaC) tool created by HashiCorp
  - It can manage resources across a wide range of providers, including AWS, Azure, GCP, and many more
- Terraform can create, update, and delete resources such as virtual machines, DNS entries, and databases
- Terraform keeps track of the current state of your infrastructure via a state file and can detect when changes need to be made to bring the infrastructure to the desired state (declarative syntax)



Source: https://developer.hashicorp.com/terraform/tutorials/aws-get-started/infrastructure-as-code

Advanced Terraform

Engineering Cloud Software Systems

Department of Software Engineering
Rochester Institute of Technology

Activity - Answer


```hcl
provider "aws" {
  region = "us-east-1"
}

locals {
  aws_key = "MZ-AWS-KEY" # Change this to your desired AWS region
}

resource "aws_instance" "my_server" {
  ami           = data.aws_ami.amazonlinux.id
  instance_type = var.instance_type
  key_name      = local.aws_key

  # Solution
  user_data     = "${file("wp_install.sh")}"               # Solution - Execute install script
  security_groups=[aws_security_group.allow_http_ssh.name]  # Solution - Specify security group for HTTP access

  tags = {
    Name = "my ec2"
  }
}
```

We'll create a new security group called '**launch-wizard-7**' with the following rules:

- [x] Allow SSH traffic from
  Helps you connect to your instance
  Anywhere
  0.0.0.0/0
- [ ] Allow HTTPS traffic from the internet
  To set up an endpoint, for example when creating a web server
- [x] Allow HTTP traffic from the internet
  To set up an endpoint, for example when creating a web server

```hcl
# Solution - Output Public IP
output "public_ip" {
  value = aws_instance.my_server.public_ip
}
```

```hcl
resource "aws_security_group" "allow_http_ssh" {
  name        = "allow_http"
  description = "Allow http inbound traffic"

  ingress {
    description = "http"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "ssh"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "allow_http_ssh"
  }
}
```

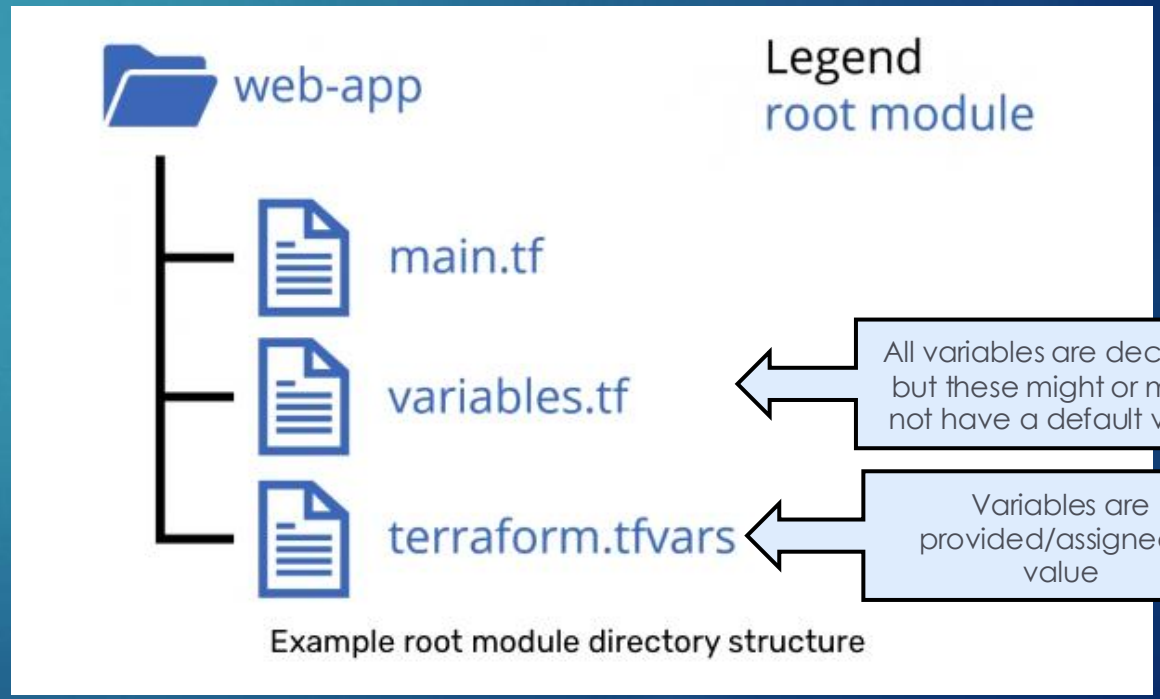
Terraform Challenges

- How can we structure Terraform code to support modular, reusable components across projects?
- How do we manage and separate infrastructure configurations for dev, test, staging, and production?
- How do we safely manage shared state using remote backends and avoid team conflicts?
- How can we securely handle sensitive values like credentials, keys, and secrets within Terraform?
- How do we manage variables consistently across Terraform code, CLI scripts, and CI/CD pipelines?

Source: https://kodekloud.com/courses/terraform-challenges/

Terraform Modules

- Modules are containers for multiple resources that are used together. A module consists of a collection of .tf and/or .tf.json files kept together in a directory
- A Terraform module allows you to create logical abstraction on the top of some resource set
- In other words, a module allows you to group resources together and reuse this group later
- Every Terraform configuration has at least one module, known as its root module, which consists of the resources defined in the .tf files in the main working directory



Example root module directory structure

Terraform Modules


```
provider "aws" {
  region = "us-east-1”
}

locals {
   aws_key = "MZ-AWS-KEY"
}

resource "aws_instance" "my_server" {
   ami           = data.aws_ami.amazonlinux.id
   instance_type = var.instance_type
   key_name      = "${local.aws_key}”
   tags = {
     Name = "my ec2"
    }
}
```

Name your resources as a best practice

main.tf

```
output "instance_ip_addr" {
  value = aws_instance.my_server[*].private_ip
}
```

output.tf

```
variable "instance_type" {
            type = string
            description = "Instance type for the EC2 instance"
            default = "t2.micro"
}
```

variable.tf

Overrides

```
instance_type="t2.small"
```

terraform.tfvars

- A .tfvars file is used to define variable values separately from the main configuration
- This file holds specific values for variables defined in the .tf configuration files, allowing you to customize settings without modifying the code.
- It’s particularly useful for managing environment-specific configurations, such as different settings for development, staging, and production

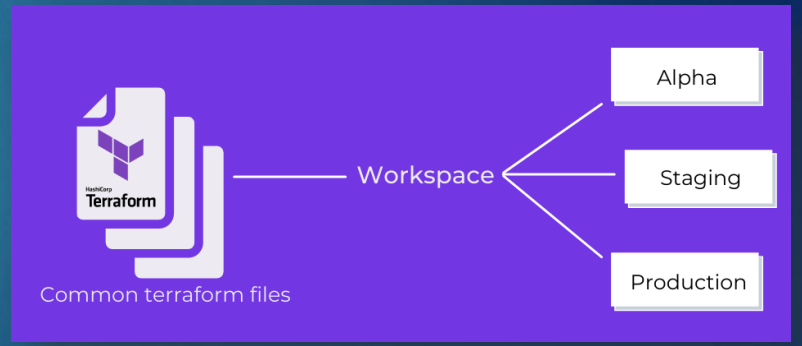
```
data "aws_ami" "amazonlinux" {
  most_recent = true
  owners = ["amazon"]

  filter {
    name="name"
    values = ["al2023-ami-2023*"]
  }
  filter {
    name="virtualization-type"
    values = ["hvm"]
  }
  filter {
    name="root-device-type"
    values = ["ebs"]
  }
  filter {
    name="architecture"
    values = ["x86_64"]
  }
}
```

data.tf

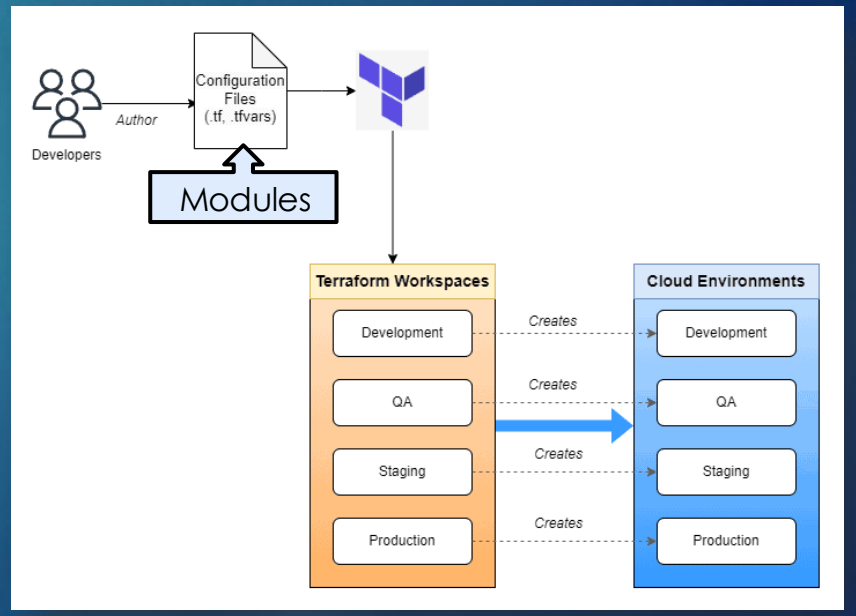
Terraform Workspaces

- Workspaces allow you to manage multiple environments (e.g., dev, staging, prod) within the same configuration
- Each workspace has its own instance of state, which means you can use the same configuration code across environments without creating conflicts in state files
- Why use them?
  - Environment Isolation
    - Easily manage separate states for different environments
  - Configuration Reusability
    - Use the same infrastructure code for each environment with separate state data
  - Simplified Workflow
    - Avoid managing different directories or configurations for each environment manually



Terraform Workspaces vs. Modules

- Modules
  - Designed for organizing and reusing code. They encapsulate resources, variables, and outputs to create reusable components (e.g., a VPC, database, or application infrastructure), allowing consistent deployments across projects
  - Designed to be reusable across multiple projects
  - You can call the same module with different variables to deploy similar resources in different environments or accounts
- Workspaces
  - They don't add functionality to the infrastructure
  - Allow reuse of the same modules without changing code, by switching the workspace, which in turn changes the associated state
  - Managed and switched using **`terraform workspace`** commands, as there's no need to modify configuration code to use them



Source: https://spacelift.io/blog/terraform-dynamodb

Terraform Workspaces - Example

- Create and switch between workspaces

```
# Create a new workspace
terraform workspace new dev

# Switch to an existing workspace
terraform workspace select dev
```

- Check which workspace you are on

```
terraform workspace show
```

- For each env, declare a .tfvars file

```
vars_dev.tfvars
vars_test.tfvars
vars_prod.tfvars
```

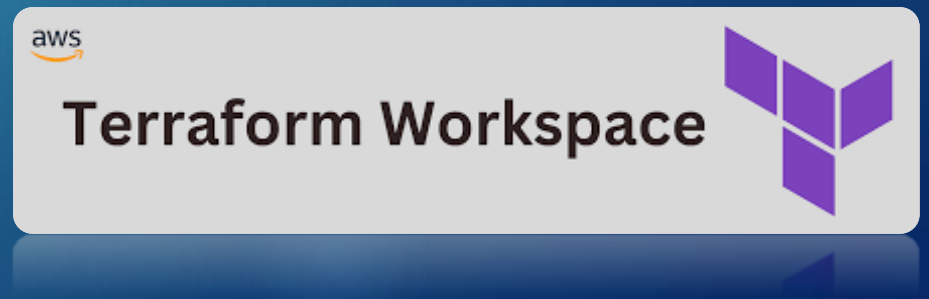
- Based on the workspace you are on (e.g. dev), you will run an apply like:

```
terraform apply –var-file=vars_dev.tfvars
```

- Another approach is to use `terraform.workspace` to set environment-specific configurations, such as EC2 instance type

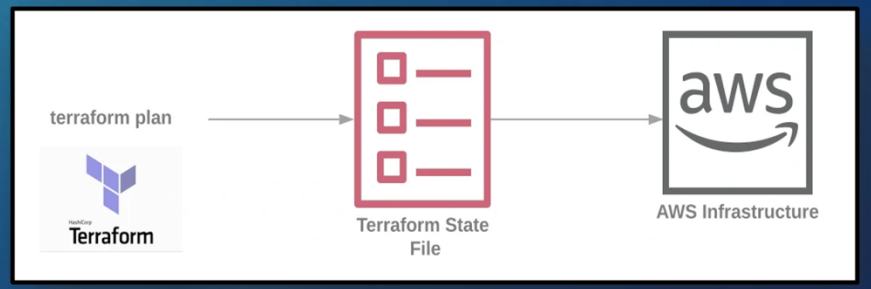
```
# Define instance type based on workspace
variable "instance_type" {
default = terraform.workspace == "prod" ? "t3.large" : "t2.micro"
}

resource "aws_instance" "example" {
ami = "ami-123456"
instance_type = var.instance_type
}
```



Terraform State Management

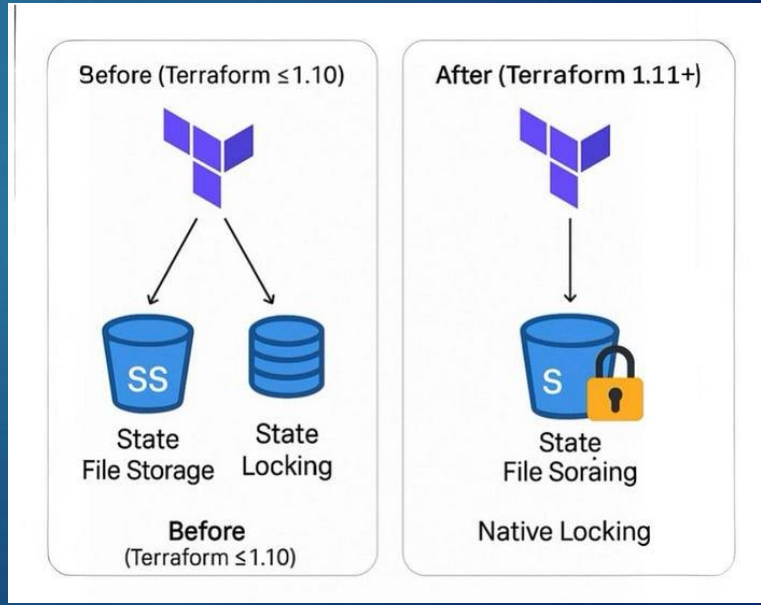
- Terraform uses a “state” file (terraform.tfstate) to keep track of infrastructure resources it manages
- The state file stores metadata that maps real-world resources to configuration, enabling Terraform to detect changes and maintain dependencies
- Why State Management is Important?
  - Resource Tracking
    - Allows Terraform to understand the current state of managed resources
  - Dependency Management
    - Manages dependencies between resources, making updates and provisioning more predictable
  - Team Collaboration
    - Remote state storage allows multiple users to share and manage the infrastructure together



Source: https://jhooq.com/terraform-manage-state/

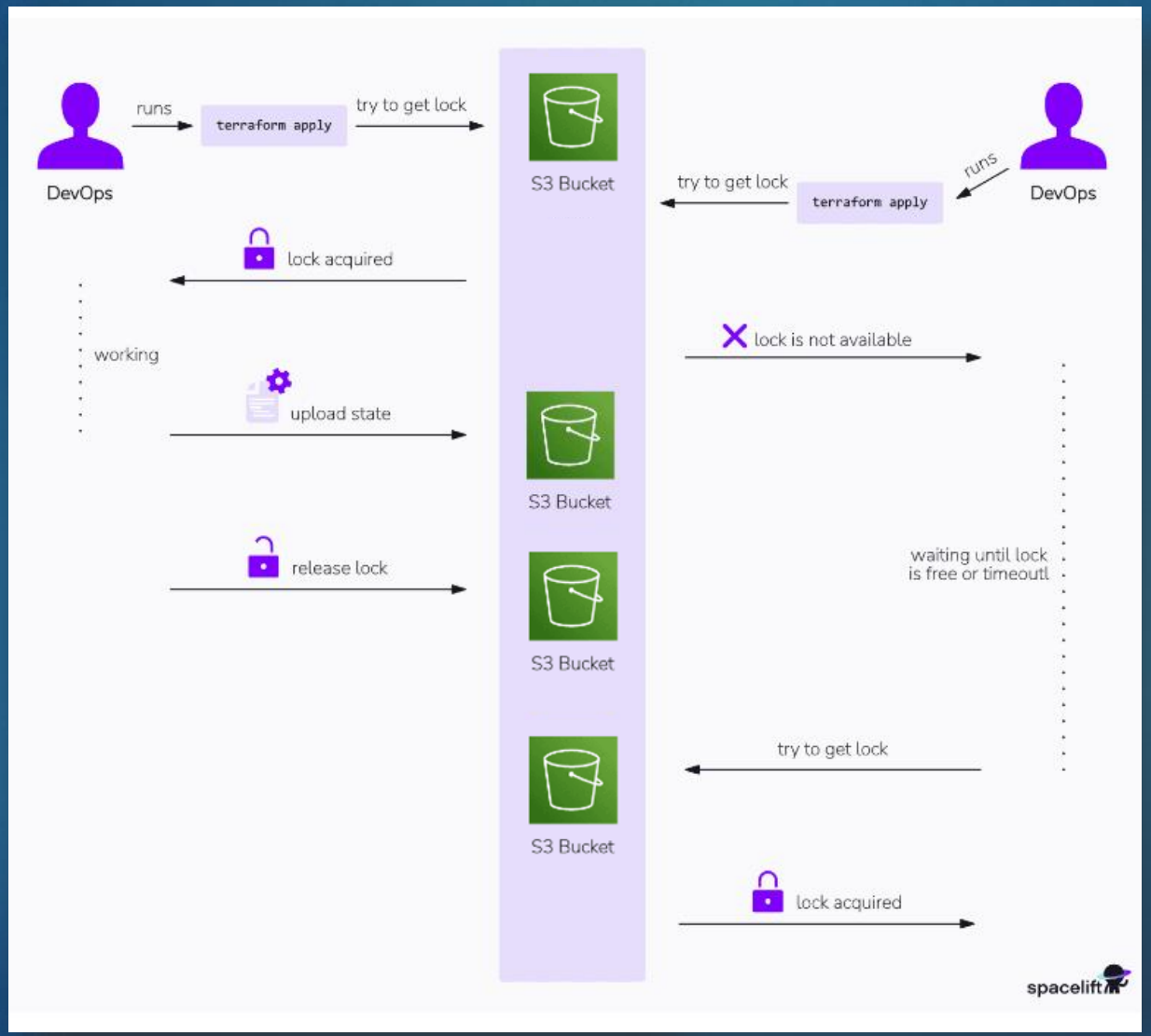
Terraform State Management

- State Storage
  - Local State
    - Stored on the user's local machine for single-user environments but can be risky if lost or corrupted
  - Remote State
    - Stored in a shared backend, such as S3, enabling collaboration and providing features like versioning, encryption, and state locking
- State Locking
  - State locking prevents multiple users from simultaneously modifying the state file, which can lead to conflicts, corruption, or resource duplication
  - DynamoDB was commonly used, but this was recently switched to S3



Source: https://medium.com/@mohamed.mourad/terraform-state-locking-migrating-from-dynamodb-to-native-s3-locking-2a49ef2668ac

Terraform State Management - Example



Source: https://spacelift.io/blog/terraform-dynamodb

Terraform State Management - Example

- Remote state configuration and locking

```
terraform {
  required_version = ">= 1.11.0"

  backend "s3" {
    bucket       = "my-terraform-state-bucket"        # S3 bucket for state storage
    key          = "prod/terraform.tfstate"           # State file path in the bucket
    region       = "us-east-1"                        # AWS region
    encrypt      = true
    use_lockfile = true
  }
}
```

If **use_lockfile = true**, Terraform creates a temporary .tflock file alongside your state object during operations, handling locking entirely within S3

- One of the most common questions that gets asked is "*how to share the Terraform state across a team?*"
  - In short, one S3 bucket should only be accessible to your team and not be made public
  - This will be covered in a future team activity

Terraform State Management - Workspaces

- Workspaces create isolated state files, allowing different instances of the same configuration to maintain separate states (useful for deploying the same infrastructure to multiple environments)

```
terraform workspace new dev
terraform apply
```

- This will create a state file for the dev workspace. On local backends, Terraform places it in the `.terraform` directory under:

  `.terraform/envs/dev/terraform.tfstate`

- In summary:
  - default workspace: `.terraform/terraform.tfstate` (if you don't use workspaces)
  - dev workspace: `.terraform/envs/dev/terraform.tfstate`
  - prod workspace: `.terraform/envs/prod/terraform.tfstate`
- Each workspace keeps its state isolated, which enables separate management of resources for different environments with the same configuration

Protecting Sensitive Input Variables in Terraform

- Terraform configurations often include sensitive information, such as database credentials, API keys, and passwords, which should be safeguarded
- Exposing sensitive information in code can lead to security vulnerabilities, particularly in collaborative or shared environments

```
resource "aws_db_instance" "wordpress_db" {
  identifier = "wordpress-db"
  allocated_storage = 20
  storage_type = "gp2"
  engine = "mysql"
  engine_version = "8.0"
  instance_class = "db.t3.micro"
  db_name = "wordpressdb"
  username = "admin"
  password = ”password123"
  parameter_group_name = "default.mysql8.0"
  skip_final_snapshot = true
  vpc_security_group_ids = [aws_security_group.rds_sg.id]
  db_subnet_group_name = aws_db_subnet_group.wordpress_db_subnet_group.name
}
```

Don't do this!

rds.tf

Protecting Sensitive Input Variables in Terraform

- Terraform supports marking variables as **sensitive**, ensuring they are not displayed in CLI output or logs
- Sensitive variables help prevent accidental exposure of confidential data during plan and apply stages
- Mark variables as sensitive
  - Use the `sensitive=true` attribute in variable definitions to keep values hidden
- Store sensitive values securely
  - Leverage secure secrets management solutions (e.g., AWS Secrets Manager, HashiCorp Vault) instead of hardcoding sensitive data

Source: https://jhooq.com/terraform-secure-sensitive-data/

Protecting Sensitive Input Variables in Terraform

- Update the Terraform configuration to use variables for the username and password, marking them as sensitive

```
variable "db_username" {
  type      = string
  sensitive = true
}

variable "db_password" {
  type      = string
  sensitive = true
}


resource "aws_db_instance" "wordpress_db" {
  identifier           = "wordpress-db"
  allocated_storage    = 20
  storage_type         = "gp2"
  engine               = "mysql"
  engine_version       = "8.0"
  instance_class       = "db.t3.micro"
  db_name              = "wordpressdb"
  username             = var.db_username
  password             = var.db_password
  parameter_group_name = "default.mysql8.0"
  skip_final_snapshot  = true
  vpc_security_group_ids = [aws_security_group.rds_sg.id]
  db_subnet_group_name   = aws_db_subnet_group.wordpress_db_subnet_group.name
}
```

rds.tf

- Create a separate file named sensitive_values.tfvars to store the actual values for `db_username` and `db_password`.

```
# sensitive_values.tfvars
db_username = "admin"
db_password = "password123"
```

sensitive_values.tfvars

- Apply the configuration by referencing the .tfvars file

```
terraform apply -var-file="sensitive_values.tfvars"
```

Protecting Sensitive Input Variables in Terraform


```
Terraform will perform the following actions:

  # aws_db_instance.wordpress_db will be created
  + resource "aws_db_instance" "wordpress_db" {
      + allocated_storage            = 20
      + db_name                      = "wordpressdb"
      + engine                       = "mysql"
      + engine_version               = "8.0"
      + identifier                   = "wordpress-db"
      + instance_class               = "db.t3.micro"
      + parameter_group_name         = "default.mysql8.0"
      + password                     = (sensitive value)
      + skip_final_snapshot          = true
      + storage_type                 = "gp2"
      + username                     = (sensitive value)
      + vpc_security_group_ids       = [
          + "sg-1234567890",
        ]
      + db_subnet_group_name         = "wordpress_db_subnet_group"
      # ... additional attributes ...
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_db_instance.wordpress_db: Creating...
aws_db_instance.wordpress_db: Creation complete after 1m34s [id=wordpress-db]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

- Sensitive values (`username` and `password`) are displayed as (sensitive value) instead of the actual values
- This prevents sensitive data from being exposed in logs or shared environments

Terraform and Variables

- Consider the following code:

```
provider "aws" {
region = "us-east-1"
}

locals {
aws_key = "MZ-AWS-KEY" # Change this to your desired AWS region
}

resource "aws_instance" "my_server" {
ami = data.aws_ami.amazonlinux.id
instance_type = var.instance_type
key_name = local.aws_key

# Solution
user_data = "${file("wp_install.sh")}" # Solution - Execute install script
security_groups=[aws_security_group.allow_http_ssh.name] # Solution - Specify security group for HTTP access

tags = {
Name = "my ec2"
}
}
```

- How could I pass in a username or environment (dev or prod) to wp_install.sh?
- This becomes more challenging with dynamic variables that are created in Terraform

Terraform and Variables – templatefile

- A **templatefile** is a Terraform function that reads a template file, fills in placeholders with variable values, and produces a formatted string output
- Used for generating dynamic content, such as configurations, scripts, and files, based on variable inputs
- Enhances readability and reusability of templates, especially for large or complex strings that include dynamic values
- Syntax Example:
  - `templatefile(path, { template variables })`
    - **path**: Location of the template file (e.g., "./file.tpl")
    - **template variables**: Map of variables to populate placeholders in the template

Source: https://fr2.readytocheckline.com/t2kf4F?ds=https://www.bitslovers.com

Terraform and Variables

```
#!/bin/bash
echo "Starting WordPress installation for ${app_environment} environment..."
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
echo "Welcome ${admin_user} to the ${app_environment} environment!" > /var/www/html/index.html
```

wp_install.tpl

- Create a new template file, wp_install.tpl, with placeholders for dynamic values
- In this template, `${app_environment}` and `${admin_user}` are placeholders to be populated by Terraform variables

```
#!/bin/bash
echo "Starting WordPress installation for production environment..."
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
echo "Welcome admin to the production environment!" > /var/www/html/index.html
```

wp_install.tpl (rendered)

- The new script with variables substituted

```
provider "aws" {
 region = "us-east-1"
}

locals {
 aws_key = "MZ-AWS-KEY"
}

variable "app_environment" {
 type    = string
 default = "production"
}

variable "admin_user" {
 type    = string
 default = "admin"
}

resource "aws_instance" "my_server" {
 ami           = data.aws_ami.amazonlinux.id
 instance_type = var.instance_type
 key_name      = local.aws_key

 # Use templatefile for dynamic user_data
 user_data = templatefile("${path.module}/wp_install.tpl", {
  app_environment = var.app_environment
  admin_user      = var.admin_user
 })

 security_groups = [aws_security_group.allow_http_ssh.name]

 tags = {
  Name = "my ec2"
 }
}
```

${path.module} refers to the directory where the Terraform module is located

ec2.tf

- The templatefile function populates `app_environment` and `admin_user` variables into the template
- Terraform then uses this customized script as user_data for the EC2 instance

Terraform Summary/Tips

- Organize Code with Modules
  - Break configurations into reusable modules for cleaner organization, modularity, and ease of maintenance across projects
- Avoid Hardcoding Variables
  - Use variables and .tfvars files to manage dynamic values, making configurations flexible and reusable across different environments.Use **format** and **validate**
- Separate Environments with Workspaces
  - Use workspaces or separate directories to manage different environments (e.g., dev, staging, prod) without mixing resources
- Limit Access to Sensitive Data
  - Mark sensitive variables (e.g., passwords, keys) as sensitive = true and avoid storing them in plain text in state files or logs
- Use terraform fmt to format all your terraform files (very handy!)
- Use terraform validate syntax and identify internal consistencies
- Plan before apply
  - Running terraform plan is the easiest way to verify if your changes will work as expected quickly

For your Term Project

- 20% of your project grade is based on how well your project environment is set up
- To earn full credit, your solution must include both setup and teardown processes implemented using Terraform
- You're encouraged to explore templates and examples online—these can be adapted to fit your specific project needs and will save your team significant time while reducing setup issues
- Mastering Infrastructure as Code (IaC) tools like Terraform is essential for anyone pursuing a career in cloud development (or DevOps)

Activity – Connect WordPress to RDS

- Do the activity in Assignments > Activity > WordPress & RDS
- You will be provided with the starter code but will need to update the code based on topics covered in this lecture